

Pizza 42 — Auth0 Architecture & Design Rationale

CIAM Specialist Tech Challenge — implementation reference and "why we picked this" guide.

1. Auth0 components used

Component	Type	Purpose
Pizza 42	Single Page Application (SPA)	The React frontend customers interact with. Initiates Universal Login, receives ID/access tokens, calls the backend API on behalf of the user.
Pizza 42 API	Custom API (Resource Server)	Logical representation of our backend in Auth0. Defines the audience (<code>https://pizza42-api</code>) and the scopes the API protects. Auth0 issues access tokens bound to this audience.
Pizza 42 Backend	Machine-to-Machine (M2M) application	Used by the serverless function to call the Auth0 Management API (read user, write <code>user_metadata</code>). Holds a client secret that lives only on the server.
Google Social Connection	Identity Provider	One-click signup/login. Pre-verified email reduces friction (no verification email needed).
Database Connection	Identity Provider	Email/password signup + Passkeys. Auth0 stores credentials so Pizza 42 never has to.
Post-Login Action	Auth0 Action	Runs on every successful login. Copies <code>user_metadata.orders</code> into the ID token as the custom claim <code>https://pizza42.com/orders</code> .

2. OAuth 2.0 grants — what we chose and what we rejected

OAuth 2.0 defines several "grants" (a.k.a. flows). Each one is a recipe for how an app obtains a token. Picking the right one is the single most important security decision in an OAuth deployment.

2.1 Authorization Code with PKCE ✓ used by the SPA

Who	Pizza 42 SPA → Auth0
How it works	Browser generates a random <i>code verifier</i> + SHA-256 <i>code challenge</i> . It redirects to Auth0 with the challenge. After login, Auth0 sends back an <i>authorization code</i> in the URL. The browser then POSTs that code + the original verifier to Auth0's token endpoint to redeem it for tokens.
Why this one	• Safe for public clients. SPAs cannot keep a client secret — anyone can view the JS. PKCE replaces the secret with a one-time crypto challenge generated per login, so even if an attacker intercepts the auth code, they can't exchange it without the verifier (which never leaves the browser). • Tokens never appear in URLs. Only the short-lived auth code is in the URL. The actual ID/access tokens come back in the body of the token endpoint response, not in browser history or referer headers. • Current OAuth 2.1 / IETF best practice. RFC 8252 and the OAuth 2.0 Browser-Based Apps BCP both mandate Auth Code + PKCE for SPAs.
Tokens received	ID token (who) + access token (what they can do at the API). Optionally a refresh token.

2.2 Client Credentials ✓ used by the backend

Who	Pizza 42 Backend (M2M) → Auth0 Management API
How it works	Backend POSTs its <code>client_id</code> + <code>client_secret</code> to Auth0's token endpoint and gets back an access token for the Management API.
Why this one	• No user is involved. The backend is acting as Pizza 42 itself to read/write user records. • Server can safely hold a secret. Unlike a SPA, the Vercel function runtime is private. • Required by the Auth0 Management API for server-to-server calls.

2.3 Implicit Grant ✗ rejected

- **Deprecated.** Returned tokens directly in the URL fragment after login — they ended up in browser history, server access logs, and referer headers.
- **No refresh token possible** (security guard rail), so apps had to re-redirect users every hour.

- **No PKCE-equivalent protection**, so a malicious browser extension or man-in-the-middle could steal tokens trivially.
- The IETF actively recommends against using it as of 2019. Auth Code + PKCE replaced it entirely.

2.4 Resource Owner Password Credentials (ROPC) **X rejected**

- **App sees the user's password.** The customer types their email + password into the Pizza 42 form, the app sends them to Auth0 in exchange for a token. This is exactly what we are trying to avoid — the security team's #1 ask was to *offload credential management*.
- **No SSO / no social login / no MFA prompts / no passkeys / no password reset link.** All those depend on Auth0 hosting the login UI.
- **OAuth 2.1 removes this grant entirely.** It's "legacy migration only".
- Using ROPC would defeat the entire purpose of choosing Auth0.

2.5 Authorization Code *without* PKCE **X rejected**

- Designed for confidential clients (servers) that can hold a secret. SPAs can't, so we'd be stuck — either embed the secret in JS (an instant credential leak) or skip the secret entirely (an authorization-code interception vulnerability).
- PKCE is the correct extension for our deployment shape.

2.6 Device Code **X not applicable**

- Designed for input-constrained devices (smart TVs, CLI tools) where typing a password is awkward. Pizza 42 is a web app with a normal keyboard — wrong tool.

2.7 Refresh Token grant **✓ Enabled with rotation**

- **Used by the SPA** (via `useRefreshTokens={true}` in `main.jsx`) to silently obtain a fresh access + ID token without re-prompting the user. Critical for the focus-listener silent renew to work reliably across Chrome, Safari, and Brave.
- **Why we chose refresh tokens over iframe silent renew:** the iframe approach relies on the Auth0 SSO cookie flowing as a third-party cookie, which modern browsers increasingly block. Refresh tokens use a back-channel POST to `/oauth/token` — no cookies involved, works everywhere.
- **Rotation + reuse detection:** each refresh issues a new refresh token and invalidates the previous one. If Auth0 ever sees the old refresh token reused, the entire token family is revoked (breach signal). Tenant-level setting.
- **Requires "Allow Offline Access"** on the API record — enabled on the Pizza 42 API in Auth0.
- **Not used by the M2M app.** The M2M Backend uses Client Credentials only — there's no user session to refresh. It re-acquires a Mgmt API token directly when its cached token nears expiry.

Why not session cookies + a traditional login form? That would mean Pizza 42 handles passwords directly — exactly the liability the security team wants to offload. Auth0 + Universal Login + OAuth removes us from PCI/credential-storage scope entirely.

3. Universal Login vs. Embedded Login

	Universal Login (used)	Embedded Login (rejected)
Where the credential form lives	Auth0's domain (<code>apnagra.us.auth0.com</code>)	Inside the Pizza 42 SPA
Pizza 42 ever sees the password	No	Yes
Social IdPs / MFA / passkeys / password reset	Free, just toggle	Each one requires custom integration
Phishing / cross-origin risk	Lower — single origin to inspect	Higher — each app can be tampered with
Branding	Customizable (logo, colors, custom domain)	Fully owned

Universal Login is Auth0's recommended default. The small loss of branding control is more than offset by the security and UX wins.

4. End-to-end token flow

1. User clicks **Log in** → SDK redirects to `https://apnagra.us.auth0.com/authorize?...&code_challenge=...`
2. Universal Login shows the form (or auto-redirects to Google if chosen).
3. After successful auth, Auth0 redirects back to `http://localhost:3000/?code=...&state=...`
4. SDK POSTs the code + PKCE verifier to `/oauth/token`, receives ID + access tokens.
5. SPA calls `POST /api/orders` with `Authorization: Bearer <access_token>`.
6. Backend verifies signature (JWKS), issuer, audience, and presence of `create:orders` scope.
7. Backend exchanges its M2M `client_id/secret` for a Mgmt API access token (Client Credentials).
8. Backend reads user → confirms `email_verified` → PATCHes `user_metadata.orders`.
9. On next login, the Post-Login Action injects `user_metadata.orders` into the ID token as `https://pizza42.com/orders`.

5. Why a separate Machine-to-Machine application?

- **Credential isolation.** The SPA cannot hold a client secret safely. The M2M app keeps the Management API secret on the server, never shipped to the browser.
- **Different trust level.** The SPA acts on behalf of one user with that user's scopes. The M2M app acts as Pizza 42 itself with admin scopes. Mixing these violates least-privilege.
- **Different grant.** SPA uses Auth Code + PKCE (user present). M2M uses Client Credentials (no user). They cannot share an application record.
- **Scope of authority.** Management API permissions (`read:users` , `update:users` , `update:users_app_metadata`) are powerful — leaking them in the browser would let an attacker modify any user.
- **Audit clarity.** Auth0 logs show which app did which action; a single mixed app would muddy investigations.

6. The three token types and what we do with each

Token	Format	Who consumes it	Used here for
ID token	JWT (OIDC)	The SPA (the client)	"Who is the user?" — name, email, picture, <code>email_verified</code> , and our custom <code>https://pizza42.com/orders</code> claim for the history view.
Access token	JWT (signed JWT, RS256)	The Pizza 42 API (backend)	"What is the user allowed to do?" — bearer credential the backend verifies and enforces <code>create:orders</code> against.
Refresh token	Opaque	Auth0 token endpoint	(Disabled in PoC.) Would let the SPA silently get a new access token after expiry without re-prompting.

7. API permissions & scope enforcement — authentication vs. authorization

Requirement #8 demands *both* a valid token **and** a specific scope. The common shortcut is to check only that the token is valid (authentication) and skip the scope check (authorization) — that's a favorite Q&A probe in identity interviews.

	Authentication (authn)	Authorization (authz)
The question	"Who are you?"	"Are you allowed to do this?"
What we check	Signature (JWKS), issuer, audience, expiry	scope claim contains create:orders
Skipping it means	Anyone can forge a token	Any logged-in user can hit any endpoint

Both checks in code (api/orders.js)

```
// Step 1 – AUTHENTICATION: "is this a real, current token from our tenant?"
const { payload } = await jwtVerify(token, JWKS, {
  issuer: `https://${AUTH0_DOMAIN}/`,
  audience: AUTH0_AUDIENCE,
});

// Step 2 – AUTHORIZATION: "is this token allowed to do THIS operation?"
const scopes = (payload.scope || "").split(" ");
if (!scopes.includes("create:orders")) {
  throw new Error("Missing required scope: create:orders");
}
```

The scope is wired in three places — miss any one and it's broken

1. **API record in Auth0** → Permissions tab declares `create:orders`. Without this, Auth0 won't include the scope on *any* token, no matter who asks.
2. **SPA (main.jsx)** → `scope: "openid profile email create:orders"` in `authorizationParams`. Without this the SPA never asks for the scope, so it's never on the token.
3. **Backend (api/orders.js)** → the explicit `scopes.includes("create:orders")` check. Without this, any valid access token — even one with no scopes — would be accepted.

Scope	Defined where	Why
create:orders	Pizza 42 API → Permissions tab	Enforces challenge requirement #8 — only access tokens explicitly granted this permission may call <code>POST /api/orders</code> . We chose a verb-noun shape so we can grow naturally: <code>read:orders</code> , <code>cancel:orders</code> , <code>refund:orders</code> .

Standard OIDC scopes also requested (not custom)

<code>openid</code>	Required by OIDC to receive an ID token at all.
<code>profile</code>	Returns <code>name</code> , <code>nickname</code> , <code>picture</code> , etc.
<code>email</code>	Returns <code>email</code> and <code>email_verified</code> — needed for the verification gate.

401 is returned if the token is:

- missing or malformed,
- not signed by our Auth0 tenant (signature fails against JWKS) — *authn fail*,
- issued for a different API (wrong `aud`) — *authn fail*,
- expired — *authn fail*,
- or lacks `create:orders` — *authz fail*.

8. RBAC vs. plain scopes — and why we chose scopes

- **Plain scopes** (what we have): the API declares `create:orders` as a permission, the SPA requests it, Auth0 stamps it into the access token's `scope` claim. Backend enforces.
- **RBAC** would mean turning on "Enable RBAC" on the API, defining roles like "customer" / "manager", assigning users to roles, and Auth0 putting their permissions into a `permissions` claim. This adds value when:
 - Different users get different sets of permissions (e.g. only staff can refund).
 - You want to manage who-can-do-what via roles instead of code.
- For this PoC every customer can place an order, so the extra layer would be ceremony without benefit. Easy to upgrade later — the backend code already checks both `scope` and `permissions` claims.

9. `app_metadata` VS `user_metadata` — right tool for each job

We deliberately use *both* metadata buckets, splitting data by who owns it.

	app_metadata	user_metadata
What we store here	Order history (orders)	Marketing profile (marketingProfile)
Who owns the data	The system / Pizza 42	The user
Who can edit it	Backend only (via M2M)	Backend or the user themselves
User-self-edit scope exists?	No	Yes (update:current_user_metadata)
Why this bucket	Orders should be tamper-proof — only the kitchen creates them.	Favorite pizza, dietary prefs, marketing opt-ins are user-owned; user should be able to edit freely.

- **Defense in depth on orders.** Even if a future engineer adds the `update:current_user_metadata` scope to let users self-edit (e.g. saved address), order history stays safe in `app_metadata` — there's no equivalent SPA scope for it.
- **Both flow into tokens easily.** A single Post-Login Action with two `setCustomClaim()` calls injects both into the ID token (under namespaced claims) so the SPA can render personalized UI on first paint.
- **Production note.** In a real Pizza 42 the source of truth for orders would be our own database, with a thin summary mirrored to `app_metadata`. Marketing profile is small enough to live entirely in `user_metadata`.

10. Auth0 Actions — and why Rules / Hooks are dead

- **Actions** (what we used): Auth0's current extensibility model. Runs Node code at specific points in the auth pipeline (Post-Login, Pre-Registration, Credentials-Exchange, etc.). API style: `api.idToken.setCustomClaim()`, `api.user.setAppMetadata()`, etc.
- **Rules** and **Hooks** are deprecated:
 - **End of Life: November 18, 2026.** All Rules and Hooks stop executing after this date.
 - **Unavailable to new tenants** created after October 16, 2023 — our trial tenant literally *cannot* use them.
 - Old API style: `context.idToken["..."] = orders;` inside a `function (user, context, callback)`.
- **Why this matters for the interview:** half of the older public Pizza 42 reference repos (PFVriens, peterpostmann, etc.) still demonstrate the Rules pattern. Cribbing from them without modernizing

is an instant red flag for an identity specialist role. Calling it out explicitly ("I noticed the older samples use Rules, which are EOL — I built this on Actions") signals you actually track the platform.

- Why we needed any extensibility at all: Auth0 strips non-OIDC claims from tokens unless code injects them. `api.idToken.setCustomClaim()` in the Post-Login trigger is the canonical way to add `https://pizza42.com/orders`.

	Rules (deprecated)	Actions (current)
API style	<code>context.idToken["claim"] = value</code>	<code>api.idToken.setCustomClaim("claim", value)</code>
Entry point	<code>function (user, context, callback)</code>	<code>exports.onExecutePostLogin = async (event, api)</code>
Runtime	Sandboxed Node 12 (frozen)	Node 18 / 22, with package management
Versioning	None — edits go live immediately	Built-in drafts and deploy
Testing	None	Built-in test runner
Available in new tenants	No (post Oct 2023)	Yes

11. Custom claim namespacing

- **The claim name we use:** `https://pizza42.com/orders` — a URI-shaped string, picked to be unique and obviously ours.
- **Why a URI:** Auth0 (following OIDC convention) reserves the unnamespaced short identifiers like `name`, `email`, `sub`, `orders` for standard or future use. Namespacing with a URI guarantees no collision.
- **The URI does *not* have to resolve.** `https://pizza42.com/orders` is never visited. It's just a globally unique string. `urn:pizza42:orders` would work equally well — convention favors HTTPS URIs.
- **The dangerous failure mode:** if you use a non-namespaced claim like `orders`, Auth0 **silently drops it**. No error, no warning, no log line — the Action runs successfully, the token is issued, the claim just isn't there. This is one of the most-time-burning bugs in the platform: "it works on my machine, but the claim isn't in the token."
- **How to debug a missing claim:** decode the ID token (jwt.io or the Debug panel in the app), confirm the claim is namespaced exactly as the SPA expects, and check Action logs in the Auth0 dashboard.

12. Email verification — why we enforce it server-side

- The SPA disables the "Place Order" button when `email_verified === false` — that's a UX hint.
- The **authority is the backend**. A hostile client could call `/api/orders` directly with a valid access token from an unverified user. The serverless function looks up `email_verified` via the Management API and rejects with 403 if false.
- We deliberately do *not* block login for unverified users (requirement #7) — they can still sign in, edit their profile, browse — they just can't order. This is good CIAM UX: don't punish users with a hard wall when a soft gate solves the same problem.

The mid-session staleness problem — and the focus-listener fix

Because `email_verified` is baked into the ID token at login time, the SPA's cached token still says `false` immediately after the user clicks the verification link in another tab. A simple page refresh doesn't fix it — the SDK just rehydrates from `localStorage`. The user would be stuck on a disabled button until they manually log out / log back in.

Fix in `App.jsx`: register a `focus` listener that triggers a silent token renewal whenever the user returns to the tab.

```
useEffect(() => {
  if (!isAuthenticated) return;
  const onFocus = () => {
    getAccessTokenSilently({ cacheMode: "off" }).catch(() => {});
  };
  window.addEventListener("focus", onFocus);
  return () => window.removeEventListener("focus", onFocus);
}, [isAuthenticated, getAccessTokenSilently]);
```

Panel talking point — verbatim:

"I added a focus listener that triggers silent re-authentication when the user returns to the tab. Auth0 issues a new ID token via a hidden iframe using the existing SSO cookie, the SDK swaps the cached token, and the UI re-renders with the current `email_verified`. The user goes from inbox → verification link → back to Pizza 42, and the button enables without any manual refresh. The backend was already always-current via the Mgmt API lookup — this is just bringing the UI freshness in line with the backend's."

- **Why this works:** `cacheMode: "off"` forces the SDK to bypass its `localStorage` cache and re-run silent auth against Auth0. The hidden iframe presents the existing SSO cookie, Auth0 reissues fresh tokens with the current `email_verified`, and the React tree re-renders with the new value.
- **Why `.catch(() => {})`:** if the user is logged out elsewhere or has a network blip, we silently ignore the failure rather than spam the console — the UI gracefully falls back to whatever was

cached.

The two silent-renew transports — iframe vs. refresh tokens

Silent renew sounds like one thing but is actually two very different mechanisms under the hood, and which one you use determines whether this whole flow works at all.

	Iframe silent renew	Refresh token silent renew
Transport	Hidden iframe → Auth0 /authorize	Back-channel POST → Auth0 /oauth/token
Depends on	Auth0 SSO cookie flowing as a <i>third-party</i> cookie	A long-lived refresh token stored client-side
Works in Chrome/Safari/Brave 2025+?	Increasingly blocked — third-party cookies are being phased out	Yes — no cookies involved at all
Requires "Allow Offline Access" on the API	No	Yes
Failure mode	Silent — iframe error swallowed	Explicit error from token endpoint

In `main.jsx` we enable refresh tokens with rotation:

```
<Auth0Provider
  ...
  cacheLocation="localStorage"
  useRefreshTokens={true}
  useRefreshTokensFallback={true}
>
```

Panel talking point — verbatim:

"Silent renew has two transports — iframe and refresh tokens. Iframe relies on the Auth0 SSO cookie flowing as a third-party cookie, which Chrome/Safari/Brave increasingly block. Refresh tokens are back-channel POST to the token endpoint — no cookies involved, works everywhere. I enabled refresh tokens with rotation: each refresh issues a new refresh token and revokes the old one, with reuse detection as a breach signal. The focus listener then reliably picks up the verified state across all browsers."

How often does the token actually refresh?

- **On tab focus** (our explicit code): every time the window/tab regains focus. Browsers de-bounce focus events to once per actual focus change, so switching tabs rapidly only fires once each direction.
- **SDK auto-renewal** (built in): the Auth0 SDK proactively renews when the cached access token is close to expiry (default access-token lifetime is 24h for SPAs unless you change it).
- **On `getAccessTokenSilently()` calls**: every order request calls this. With `cacheMode: "on"` (default) it returns the cached token if still valid; `cacheMode: "off"` forces a fresh round-trip.

Net effect: on every tab focus we force a fresh round-trip; otherwise the SDK manages things based on token expiry. There's no continuous polling — refreshes are event-driven, not interval-based, which keeps Auth0 API usage low while keeping the UI feeling current.

Refresh token rotation in particular: each successful refresh returns a *new* refresh token and invalidates the previous one. If Auth0 ever sees the old refresh token reused, that's a strong breach signal — the entire token family is revoked and the user is forced to log in again. Adds friction in a leak scenario, which is exactly what you want.

13. Security summary — defense in depth

- **Universal Login** — Auth0 hosts the credential prompt; Pizza 42 never sees passwords.
- **Auth Code + PKCE** — current best practice for SPAs.
- **JWT with RS256** — asymmetric crypto, backend verifies via JWKS, no shared secrets to leak.
- **Issuer + audience + scope + expiry + signature checks** — all enforced on every API request.
- **Email verification enforced server-side.**
- **M2M client secret stored as Vercel env var**, never shipped to the browser.
- **Token caching** — the Mgmt API token is cached in memory until shortly before expiry, reducing token-endpoint traffic.
- **Built-in Auth0 protections** — MFA, breached password detection, bot detection, anomaly detection, brute-force protection — all toggleable with zero code changes.

14. What I'd add for a real production rollout

- Pizza 42's own DB as source of truth, mirroring a summary into `user_metadata`.
- Refresh token rotation re-enabled for longer sessions.
- MFA + breached password detection turned on.
- A custom Auth0 domain (`auth.pizza42.com`) for branding and cookie scoping.
- Step-up auth on higher-risk actions (e.g. adding payment, large orders).

- Account linking so a user who signs up with email and later uses Google ends up as one identity, not two.
- Progressive profiling via a Post-Login Action — capture marketing data (favorite pizza, delivery address) gradually instead of in one big signup form.

15. PDF requirements that are NOT production best practice

Talking thoughtfully about where the spec departs from real-world best practice is a key signal to the panel — disagreeing with the brief, when justified, is encouraged.

15.1 "Add order history to the ID token" (req #10)

- **Anti-pattern.** ID tokens should be small and stable.
- Tokens travel in headers/cookies — unbounded arrays bloat them fast.
- Tokens contain potentially sensitive data with a 24h+ lifetime — exposing full purchase history widens the blast radius if a token leaks.
- Re-fetched on every silent renewal, wasting bandwidth.
- **Better:** include a *summary* in the token (`orderCount` , `lastOrderAt` , `topItem`) and fetch the full list on demand via `GET /api/orders` protected by a `read:orders` scope.

15.2 "Save the order to the user's Auth0 profile" (req #9)

- **Doesn't scale.** `user_metadata` is capped at 16 KB per user — months of orders blow through that.
- Not queryable — you can't answer "what did everyone order this week?".
- Couples customer data to your identity vendor (lock-in).
- Management API is rate-limited; a busy hour could exhaust quotas.
- **Better:** own DB is source of truth. Mirror a thin summary to `user_metadata` only if needed in tokens.

15.3 Calling the Mgmt API on every order

- What our PoC does for simplicity. In production you'd:
 - Use a Post-Login Action or Auth0 webhook to update derived state in your own systems instead of round-tripping through Mgmt API on the hot path.
 - Cache aggressively (we already cache the M2M access token in-memory).
 - Batch updates where possible.

15.4 Implicit framing of "verify email blocks ordering"

- PDF could be read as a client-side gate. We implemented it **server-side** in the API handler. The client-side disable is just UX — the API is the authority.
- A hostile client could call `POST /api/orders` directly with a valid token from an unverified user. Only server-side enforcement closes that hole.

15.5 Audience as a URL

- We chose `https://pizza42-api` by convention, but the OAuth spec allows any URI. `urn:pizza42:api` is arguably cleaner because it's obviously not a resolvable endpoint and won't confuse readers.

16. Panel interview — questions to prepare for

Direct quotes / paraphrases from the recruiter call. Each one needs a 1–2 sentence answer ready to go.

Question	Where to find the answer
"Why Auth Code + PKCE, not Implicit?"	Section 2.1 + 2.3 of this doc — tokens not in URL, PKCE replaces secret, current best practice.
"Why a separate M2M application?"	Section 5 — credential isolation, different grant, least privilege.
"What about refresh tokens?"	Section 2.7 + token-types table — we're using silent iframe renewal; would re-enable with rotation in production.
"Back channel vs front channel?"	Front channel = browser redirects (authorize endpoint). Back channel = server-to-server (token exchange, JWKS fetch, Mgmt API calls). Tokens are issued via the back channel so they never appear in the URL.
"Why save orders to <code>user_metadata</code> / put them in the ID token?"	Use the critique in section 15 — be ready to disagree with the spec and explain the trade-off.
"How would you do progressive profiling?"	Post-Login Action that prompts for marketing data (favorite pizza, delivery zone) over multiple visits instead of in one huge signup form. Stored in <code>user_metadata</code> . Directly addresses Pizza 42 marketing team's ask.
"What's the GenAI angle?"	Mentioned as bonus by the recruiter. Suggest a chatbot that uses order history claim to recommend reorders, plus Auth0's emerging Auth for GenAI (delegated tool use, agent identity).
"What changes at 10M users?"	Move orders to own DB, derive summary into metadata only if used in tokens, custom domain for cookie scoping, refresh-token rotation, MFA conditional rules, observability via Auth0 log streams.
"How do you rotate the M2M client secret?"	Auth0 supports multiple credentials per M2M app — generate the new one, deploy to env vars, rotate traffic, revoke old. For higher assurance: switch M2M to private-key JWT (mTLS-style auth) instead of shared secret.
"What if Auth0 is down?"	Frontend gracefully degrades to read-only / cached views. JWT validation works locally as long as we cached the JWKS. New logins are blocked — there's no realistic mitigation other than a multi-IdP fallback (a bigger architectural change with its own cost).

17. Panel demo plan (45 minutes)

Recruiter said: 25 min demo, 10 min walkthrough, 10 min Q&A. Audience is mixed: marketing/PM stakeholders + technical engineers.

Slides (~5 min)

1. **Pizza 42 business goals** — security wants to offload credentials, product wants frictionless login, marketing wants enriched profiles. Map each to a Auth0 capability.
2. **Architecture diagram** — SPA / Pizza 42 API / M2M backend / Mgmt API / Action.
3. **OAuth flow** — front channel (redirect) vs back channel (token exchange + Mgmt API).
4. **Token claims** — ID vs access vs (disabled) refresh; what each contains.

Live demo (~20 min)

1. Land on hero page (logged out) — call out Universal Login button.
2. Sign up with email/password — show `email_verified: false` in debug payload, banner blocks ordering.
3. Verify email via link — refresh, banner disappears.
4. Place an order — show backend log (scope verified, Mgmt API patched), show success response.
5. Log out, log back in — order now appears in history (custom claim).
6. Log in via Google instead — pre-verified, one-click.
7. Open Auth0 dashboard — show API + scope, Post-Login Action code, user record's `user_metadata.orders`.

Walkthrough (~10 min)

- Tech stack: React + Vite, Vercel serverless, `jose` for JWT, Auth0 SDK.
- Implementation decisions: Universal Login over embedded, scope-based authz over RBAC, M2M for Mgmt API.
- What I disagreed with from the spec (section 15).
- Challenges: per-app authorization config, localhost consent prompt, ID-token claim namespacing.
- Production roadmap (section 14).

Q&A (~10 min)

Field questions using section 16. Don't bluff — if I don't know, say so and explain how I'd find out.

18. Mapping Pizza 42 business goals to Auth0 features

Useful for the business-stakeholder portion of the demo.

Pizza 42 ask	Auth0 capability	Where in this PoC
Security: offload credential management	Universal Login + Database connection	SPA never sees passwords; Auth0 hashes & stores them.
Security: storing credentials raises liability	Auth0 is SOC 2 / ISO 27001 / GDPR-ready	Compliance burden shifted to Auth0.
Product: frictionless & customizable login	Universal Login pages, brandable	Tenant has the Auth0 New Universal Login experience.
Product: turnkey password reset	Built into Universal Login	Zero app code; reduces help-desk tickets.
Product: social login	Google social connection	Enabled in tenant; demo shows one-click signup.
Product: not remember password	Passkeys (WebAuthn) + Passwordless	Enabled on database connection; key talking point.
Marketing: enrich customer data	Social IdP claims pre-populate name/picture/locale; marketing profile UI captures favorite item, dietary prefs, opt-ins into <code>user_metadata</code> ; behavioral data accumulates in <code>app_metadata.orders</code>	"Tell us about you" card writes to <code>/api/profile</code> → <code>user_metadata.marketingProfile</code> ; surfaced in ID token via Post-Login Action.